

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – MINI APPLICATION DEVELOPMENT (CHATBOT / SUMMARIZER / TRANSLATOR)

Course Code:	CSA6503	Course Title:	Generative AI and Large Language Models
CO Assessed:	CO4 – Precision (BL3, BL4,BL5)	Assessment:	Assessment Tool 1-Mini Application Development (Chatbot / Summarizer / Translator)
Weightage:	20% of CIA	Total Marks:	2 * 50 = 100
CO4 Statement:	<i>Design and implement multimodal Generative AI applications integrating text, image, and speech models for engineering applications.(BL3, BL4,BL5)</i>		
Student Name:	M.Someswar Reddy	Reg. No.:	192472081
Section / Batch:	CSE-AI	Date:	20/8/26

Instructions to Students

- Answer any 2 questions. Each question carries 50 marks. Total: 100 marks.
 - Implement the solution using Python.
 - Select and justify an appropriate AI/LLM model or technique.
 - Develop a working application with a user interface.
 - Test the application using multiple input cases.
 - Demonstrate handling of invalid, empty, or unexpected inputs.
 - Clearly explain the application architecture and workflow.
 - Submit the source code, test results, screenshots/demo, and documentation.
- Explain the limitations of the selected model/technique and suggest possible improvements.

Code of Conduct

*I **M.Someswar Reddy/192472081** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: **M.Someswar Reddy**

QUESTION 6 – COURSE INFORMATION CHATBOT

Q6 asks for a chatbot that answers questions about a selected course, including syllabus, units, learning outcomes, prerequisites, assessment methods, and laboratory activities.

Aim

To develop an AI-powered Course Information Chatbot that answers student questions using the supplied course information as its knowledge source.

Objectives

- Provide a simple Streamlit chat-style interface.
- Load course information from a local text file.
- Use a Gemini large language model to generate answers.
- Restrict answers to the supplied course material.
- Provide a fallback response when information is unavailable.
- Handle empty questions.
- Test the chatbot using course-related and unrelated questions.

Technologies

- Python
- Streamlit
- Google GenAI Python SDK
- Gemini large language model
- TXT course-information knowledge file

Project Structure

Q6_Course_Information_Chatbot/

```
|— app.py
|— course_data.txt
|— .streamlit/
    |— secrets.toml
```

Workflow

1. Start the Streamlit application.
2. Load the Gemini API key from Streamlit secrets.
3. Read course_data.txt.
4. Accept the student's question.
5. Build a prompt containing the course material and question.
6. Send the prompt to Gemini.
7. Display the generated answer.
8. If the answer is not supported by the course material, display the fallback response.

Source Code:

```
import streamlit as st
from google import genai

st.set_page_config(
    page_title="Course Information Chatbot",
    page_icon=""
)

st.title("Course Information Chatbot")
st.write("Ask questions about your course.")

api_key = st.secrets["GEMINI_API_KEY"]
client = genai.Client(api_key=api_key)

with open("course_data.txt", "r", encoding="utf-8") as file:
    course_data = file.read()

question = st.text_input("Enter your question:")

if st.button("Ask"):

    if question.strip() == "":
        st.warning("Please enter a question.")

    else:
        prompt = f"""
You are a Course Information Chatbot.

Answer the student's question using ONLY
the course information below.

COURSE MATERIAL:
{course_data}

STUDENT QUESTION:
{question}

Instructions:
1. Give a clear and concise answer.
2. Do not invent or guess information.
3. If the information is not present, say:
"Sorry, the information is not available in the course material."
4. Answer only questions related to the course material.
```

5. Use simple language suitable for a college student.

"""

```
try:
    response = client.models.generate_content(
        model="gemini-3.6-flash",
        contents=prompt
    )
```

```
    st.subheader("Answer")
    st.write(response.text)
```

```
except Exception as e:
    st.error("Unable to generate the answer.")
    st.write(str(e))
```

OUTPUT

Deploy ⋮

Course Information Chatbot

Ask questions about your course.

Enter your question:

What is CO4?

Ask

Answer

CO4 is: Design and implement multimodal Generative AI applications integrating text, image, and speech models for engineering applications.



Course Information Chatbot

Ask questions about your course.

Enter your question:

What is the course code?

Ask

Answer ↔

The course code is CSA65.

Knowledge File

The knowledge file contains the course and assessment information used by the chatbot. The chatbot is instructed not to invent information that is absent from this file.

COURSE INFORMATION

Course Name:

Generative AI and Large Language Models

Course Code:

CSA65

QUESTION 6:

Develop a Course Information Chatbot.

The chatbot should answer questions about:

- Syllabus
- Units
- Learning outcomes

- Prerequisites
- Assessment methods
- Laboratory activities.

Result:

The Course Information Chatbot successfully runs in Streamlit, connects to Gemini, reads the course knowledge file, accepts questions, and displays generated answers. It also uses a controlled prompt and fallback response to reduce unsupported answers.

Limitations and Future Improvements

- The answer quality depends on the completeness of course_data.txt and the selected LLM.
- The current knowledge source is a local text file.
- The application requires Gemini API access.
- Future versions can use a PDF/document loader and retrieval-based RAG.
- More course-specific test questions can be added for evaluation.

QUESTION 15 – NEWS ARTICLE SUMMARIZER

Q15 requires a text summarization application that accepts a long news article, generates a concise summary, provides a simple interface, and allows the user to control the approximate summary length.

Aim

To develop an AI-powered News Article Summarizer using a 50-article dataset, Streamlit, and Gemini, with Short, Medium, and Detailed summary-length controls.

Objectives

- Maintain a dataset of 50 different practice news-style articles.
- Allow the user to select an article.
- Display the selected article content.
- Generate a concise AI summary.
- Provide Short, Medium, and Detailed summary options.
- Preserve important facts and avoid invented information.
- Handle dataset and API errors.

Technologies

- Python
- Streamlit
- Pandas
- Google GenAI Python SDK
- Gemini large language model
- CSV dataset

Project Structure

Q15_NEWS_ARTICLE_SUMMARIZER/

```
|— app.py
|— .streamlit/
|   |— secrets.toml
|— dataset/
|   |— news_articles_50.csv
```

Dataset

The dataset contains 50 different original practice news-style articles. Each record contains an ID, category, title, and article text. The CSV is used by Pandas and the Streamlit application.

Workflow

1. Start the Streamlit application.
2. Load the Gemini API key from Streamlit secrets.
3. Read news_articles_50.csv using Pandas.
4. Display the number of loaded articles.
5. Allow the user to select an article.

6. Display the selected article.
7. Allow the user to choose Short, Medium, or Detailed.
8. Create a controlled summarization prompt.
9. Send the article and instruction to Gemini.
10. Display the AI-generated summary.

Source Code

```
import streamlit as st
import pandas as pd
from google import genai

st.set_page_config(
    page_title="News Article Summarizer",
    page_icon="",
    layout="wide"
)

st.title(" News Article Summarizer")
st.write("Select a news article from the dataset and generate an AI summary.")

api_key = st.secrets["GEMINI_API_KEY"]
client = genai.Client(api_key=api_key)

dataset_path = "dataset/news_articles_50.csv"

try:
    df = pd.read_csv(dataset_path)
except Exception as e:
    st.error("Unable to load the news dataset.")
    st.write(str(e))
    st.stop()

st.success(f"Dataset loaded successfully: {len(df)} articles")

article_number = st.selectbox(
    "Select a News Article",
    range(len(df)),
    format_func=lambda x: f"{x + 1}. {df.iloc[x]['title']}"
)

selected_article = df.iloc[article_number]
```

```
st.subheader(" Selected Article")
st.write(f"**Category:** {selected_article['category']}")
st.write(f"**Title:** {selected_article['title']}")
```

```
article_text = selected_article["article"]
```

```
st.text_area(
    "Article Content",
    article_text,
    height=300
)
```

```
summary_length = st.selectbox(
    "Select Summary Length",
    ["Short", "Medium", "Detailed"]
)
```

```
if st.button("Summarize Article"):
```

```
    if not article_text or len(article_text.strip()) == 0:
        st.warning("The selected article is empty.")
```

```
    else:
        if summary_length == "Short":
            instruction = "Summarize in 2 to 3 sentences."
        elif summary_length == "Medium":
            instruction = "Summarize in 4 to 6 sentences."
        else:
            instruction = (
                "Give a detailed summary covering the main points, "
                "important facts, and conclusion."
            )
```

```
        prompt = f"""
You are an AI News Article Summarizer.
```

```
Summarize the following news article.
```

```
ARTICLE:
{article_text}
```

```
SUMMARY LENGTH:
{summary_length}
```

INSTRUCTION:

{instruction}

Rules:

1. Preserve important information.
2. Do not invent facts.
3. Do not add information that is not present in the article.
4. Use clear and simple language.
5. Provide only the summary.

"""

try:

```
response = client.models.generate_content(  
    model="gemini-3.6-flash",  
    contents=prompt  
)
```

```
st.subheader("AI Generated Summary")  
st.write(response.text)
```

except Exception as e:

```
st.error("Unable to generate the summary.")  
st.write(str(e))
```

OUTPUT:



News Article Summarizer

Select a news article from the dataset and generate an AI summary.

Dataset loaded successfully: 50 articles

Select a News Article

1. AI assistants improve small-business productivity



Selected Article

Category: Technology

Title: AI assistants improve small-business productivity

Article Content

AI assistants improve small-business productivity



News Article Summarizer

Select a news article from the dataset and generate an AI summary.

Dataset loaded successfully: 50 articles

Select a News Article

1. AI assistants improve small-business productivity



Selected Article

Category: Technology

Title: AI assistants improve small-business productivity

Article Content

AI assistants improve small-business productivity

The initiative illustrates a broader effort to combine practical planning with data-based decision-making. Supporters believe that careful testing can help organizations adopt useful changes while avoiding unnecessary risks. The project will continue to be monitored before any larger expansion is considered.

Select Summary Length

Short

 Summarize Article



AI Generated Summary

A technology sector initiative is testing practical, data-driven methods to improve everyday services while keeping costs and complexity low. Early testing shows that better planning and monitoring can yield positive results, though organizers stress that further testing is needed before drawing final conclusions. Future phases will evaluate findings across multiple locations to create a comprehensive report and determine if the project should be expanded.

Result

The News Article Summarizer successfully loads the 50-article CSV dataset, allows article selection, provides summary-length choices, sends the article to Gemini, and displays the generated summary. The Short summary was demonstrated successfully. Medium and Detailed should be captured as final test screenshots before submission.

Combined Testing and Demo Checklist

- Q6 chatbot main screen screenshot.
- Q6 course-related question and answer screenshot.
- Q6 fallback/unsupported question screenshot.
- Q15 dataset loaded screenshot.
- Q15 article selected screenshot.
- Q15 Short summary screenshot.
- Q15 Medium summary screenshot.
- Q15 Detailed summary screenshot.
- Q15 invalid/empty input handling screenshot.
- Final application run screenshots.

Conclusion

Q6 and Q15 together demonstrate two Generative AI mini applications: a Course Information Chatbot and a News Article Summarizer. Both are implemented in Python with Streamlit and Gemini. Q6 uses a course-information knowledge file and controlled fallback behavior, while Q15 uses a 50-article CSV dataset and summary-length control. The applications are structured for demonstration, testing, and documentation according to the assessment requirements.